



**MULUNGUSHI UNIVERSITY**  
**SCHOOL OF ENGINEERING AND TECHNOLOGY**

**BACHELOR OF SCIENCE IN COMPUTER SCIENCE**

# **ICT461 — Web Standards and HTTP Fundamentals**

**NAME: JENNIPHER MWENGE**

**STUDENT No: 202301743**

**COURSE: WEB SYSTEMS AND TECHNOLOGY**

**CODE: ICT 461**

**LECTURER: MR.NYIRENDA**

## Task 4, Item 10 — Cookie Security, CSRF, and Response Header Audit

*Course Registration Portal — Mulungushi University*

### 1. Why production session cookies need Secure over HTTPS

Without the Secure flag, a cookie can be sent over both HTTPS and plain, unencrypted HTTP. If any part of a session ever happens over HTTP even a single stray request, or a user typing the address without "https://" , the cookie travels across the network as plain text. Anyone sharing that network (public Wi-Fi, a compromised router, a network operator) could intercept it and reuse it to impersonate that user's session. The Secure flag forces the browser to withhold the cookie unless the connection is HTTPS, removing that exposure entirely.

### 2. Why HttpOnly limits JavaScript access

By default, JavaScript running on a page can read cookies through `document.cookie`. `HttpOnly` removes the cookie from what `document.cookie` exposes: the browser still attaches it automatically to outgoing requests, but page scripts can no longer read or copy it. This matters because if an attacker manages to inject malicious JavaScript into the page (a cross-site scripting, or XSS, attack), `HttpOnly` prevents that injected script from directly stealing the session cookie.

### 3. Why SameSite does not replace all CSRF protection

`SameSite=Lax` stops the cookie from being attached to most cross-site requests for example, a hidden form on another website that auto-submits a POST to your site no longer carries the cookie. This blocks a large share of CSRF attacks.

However, `Lax` still allows the cookie on top-level navigations. If a malicious site tricks a user into clicking a link that triggers a GET request with a side effect (changing a setting, for instance), the cookie still travels with it. `SameSite` reduces the attack surface considerably but does not close every path, so it functions as one layer alongside other protections such as CSRF tokens, rather than a complete replacement for them.

### 4. Cookie sessions compared with bearer tokens

Cookie sessions: the browser attaches the cookie to every matching request automatically, with no code required on the client side. This is convenient, but that automatic attachment is exactly what makes CSRF possible , the browser does not check whether the user intended to send it on a given request.

Bearer tokens: the client's own JavaScript must explicitly attach the token to a header (for example, `Authorization: Bearer <token>`) on every request. This avoids CSRF, because a malicious site cannot make the browser attach something it has no access to. The trade-off is that the developer must handle storing and attaching the token correctly, and a token made accessible to JavaScript (for example, stored in `localStorage`) becomes vulnerable to theft through XSS instead.

### 5. Response header audit

Site inspected: <https://owasp.org>

Method: opened the site in Microsoft Edge, pressed F12, opened the Network tab, reloaded the page, selected the first request (owasp.org, the main document, GET /), and read the Response Headers panel.

| Header                    | Present / Absent | Notes                                                                           |
|---------------------------|------------------|---------------------------------------------------------------------------------|
| Content-Security-Policy   | Absent           | Not among the response headers returned for the main document request.          |
| Strict-Transport-Security | Absent           | Not observed in this response, though the site is served over HTTPS by default. |
| X-Content-Type-Options    | Absent           | Not observed in this response.                                                  |
| Referrer-Policy           | Absent           | Not observed in this response.                                                  |

Headers actually observed on this response: Cache-Control (public, max-age=0, must-revalidate), Date, Server (Vercel), X-Vercel-Cache, X-Vercel-Id.

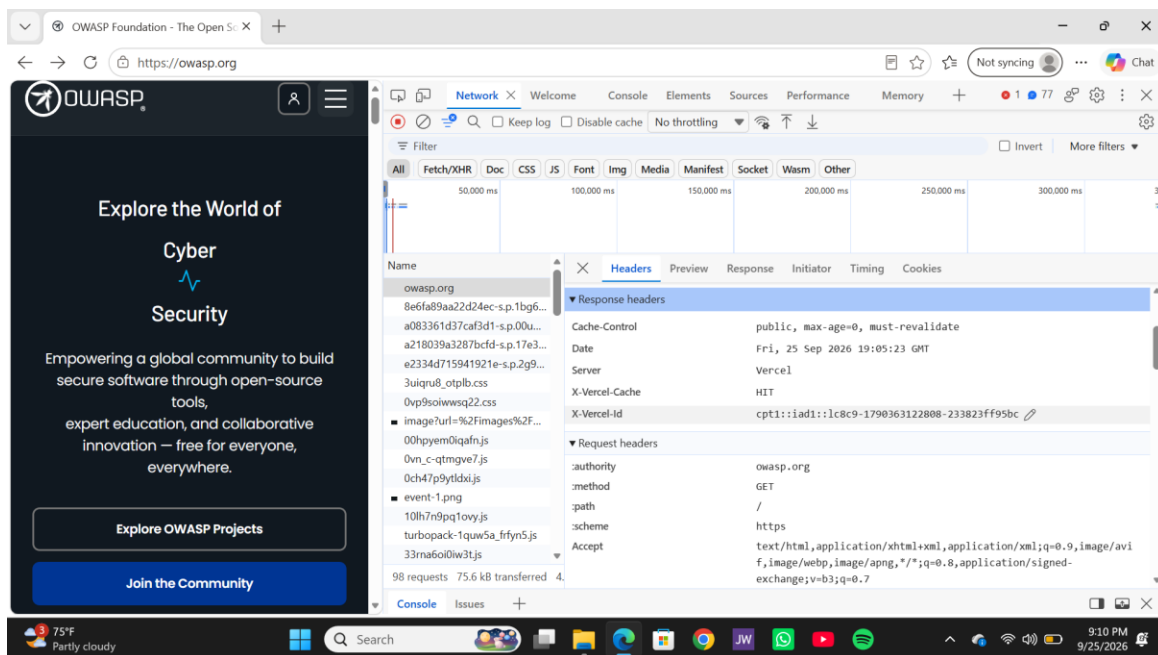


Figure 1. DevTools Network panel, Response Headers for https://owasp.org (main document request).